

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_10

10. Adapter Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Adapter pattern.

GoF Definition

It converts the interface of a class into another interface that clients expect. The Adapter pattern lets classes work together that could not otherwise because of incompatible interfaces.

Concept

From the GoF definition, you can guess that this pattern deals with at least two incompatible inheritance hierarchies. In a domain-specific system, the clients know how to invoke methods in software. These methods can follow an inheritance hierarchy. Now assume that you need to upgrade your system and need to follow a new inheritance hierarchy. When you do that, you do not want to force your clients to learn the new way to access the software. So, what can you do? The solution is simple: you write an adapter that accepts client requests and translates these requests into a form that the methods in the new hierarchy can understand. As a result, clients can enjoy the updated software without any hassle. By using this pattern, you make two incompatible interfaces compatible so that they can work together.

There are two types of adapters: class adapters and object adapters. I'll cover both types in this chapter.

Real-Life Example

A common use of this pattern is when you use an electrical outlet adapter/AC power adapter on international travel. These adapters act as middlemen so that an electronic device, say a laptop that accepts a U.S. power supply, can be plugged into a European power outlet.

Even a translator who converts one language to another can be considered to be following this pattern in real life.

Let's consider a situation where you have two different shapes (say Shape1 and Shape2), neither of which is a rectangle, and they look like those in Figure [10-1](#).

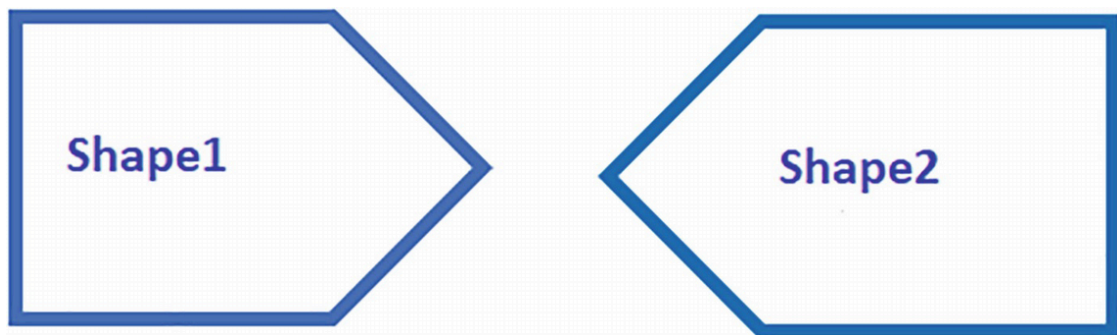


Figure 10-1 Before using an adapter

Let's further assume that you need to form a rectangle by combining these two shapes. How can you proceed? Ok, one simple solution is to bring in another bounded X-shaped figure like the one shown in Figure [10-2](#) (which is filled with colors here).



Figure 10-2 An adapter

Then you can attach these three shapes as shown in Figure [10-3](#).

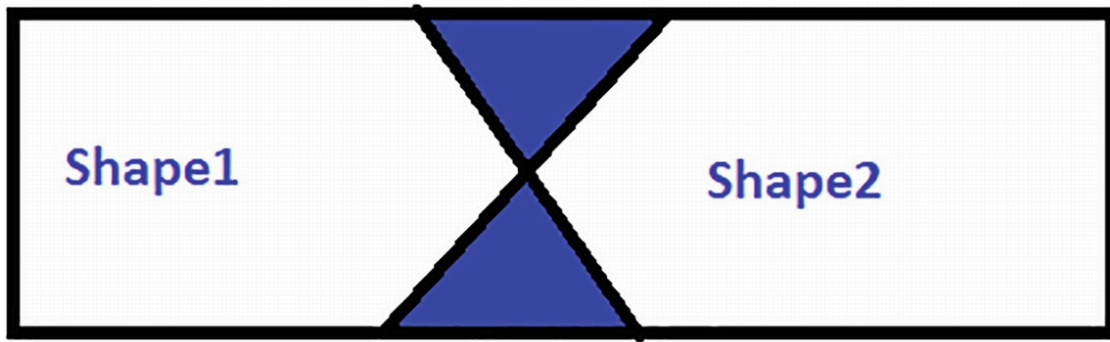


Figure 10-3 After using an adapter

In programming, you can think of Shape1 and Shape2 as two different interfaces that can't work together unless you combine them to form a rectangle using this X-shaped figure. **The X-shaped figure is playing the role of an adapter in this scenario.**

Computer World Example

Suppose you have an application that can be broadly classified into two parts: the user interface (UI or the front end) and the database (the back end). Through the user interface, clients can pass some specific type of data or objects. Your database is compatible with those objects and can store them smoothly. Over a period of time, you may realize that you need to upgrade your software to make your clients happy. So, you may want to allow some other type of object to also pass through the UI. But in this case, the first issue will come from your database because it cannot store the new type of object. In such a situation, you can use an adapter that will take care of the conversion of these new objects to a compatible form that your existing database can accept and store. Here is another example for you:

- In Java, you can consider the `java.io.InputStreamReader` class and the `java.io.OutputStreamWriter` class as examples of this pattern. They adapt an existing `InputStream/OutputStream` object to a `Reader/Writer` interface. They are object adapters. You will learn about class adapters and object adapters shortly.

Note

`Reader` and `Writer` are abstract classes in Java. When we use polymorphic code, we often use the term “interface” as a supertype, but not as a Java interface. So, you should not assume that the term “interface” is limited to Java interfaces only.

Implementation

In the upcoming example, there are two hierarchies, one for rectangles and one for triangles. There are two methods in the interface called `RectInterface`: `calculateArea()` and `aboutMe()`. The `Rectangle` class implements this interface and forms the first hierarchy as follows:

```
package jdp3e.adapter;

interface RectInterface {
    void aboutMe();
    double calculateArea();
}

class Rectangle implements RectInterface {
    double length;
    public double width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    @Override
    public void aboutMe() {
        System.out.println("Shape type: Rectangle.");
    }
    @Override
    public double calculateArea() {
        return length * width;
    }
}
```

The `TriInterface` interface has two methods called `calculateTriangleArea()` and `aboutTriangle()`. The `Triangle` class implements this interface and forms another hierarchy as follows:

```
interface TriInterface {
    void aboutTriangle();
    double calculateTriangleArea();
}

class Triangle implements TriInterface {
    double baseLength;
    double height;
    public Triangle(double length, double height) {
```

```

        this.baseLength = length;
        this.height = height;
    }
    @Override
    public double calculateTriangleArea() {
        return 0.5 * baseLength * height;
    }
    @Override
    public void aboutTriangle() {
        System.out.println("Shape type: Triangle.");
    }
}

```

These two hierarchies are easy to understand. Now, suppose **you need to calculate the area of a triangle using the Rectangle hierarchy**.

How can you proceed? Yes, you guess it right: you need to use an adapter to solve this problem. Here is an example:

```

/*
 * Adapter is implementing RectInterface.
 * So, it needs to implement
 * all the methods defined
 * in the target interface.
 */
class Adapter implements RectInterface {
    TriInterface triangle;
    public Adapter(TriInterface triangle) {
        this.triangle = triangle;
    }
    @Override
    public void aboutMe() {
        triangle.aboutTriangle();
    }
    @Override
    public double calculateArea() {
        return triangle.calculateTriangleArea();
    }
}

```

Notice the beauty of using the adapter. You are not making any changes to any hierarchy. By using an `Adapter` instance, you use the methods of

`RectInterface` to calculate the area of a triangle. It is because although you are using the `aboutMe()` and `calculateArea()` methods of the `RectInterface` at a high level, inside those methods you are invoking the methods of `TriInterface`.

NoteIn this example, the `Adapter` adapts `TriInterface` to the `RectInterface`, so the client can use the incompatible `TriInterface` methods in the `RectInterface` hierarchy.

Apart from this advantage, you can also extend the benefit of using an adapter. For example, suppose you need to have a large number of rectangles in an application, but there is a constraint on the number of rectangles you can create. (For simplicity, let's assume that you are allowed to create a maximum of 5 rectangles and 10 triangles, but when the application runs, in certain scenarios, you may need to supply 10 rectangles.) In those cases, using this pattern, you can use some of the triangle objects that can behave like rectangle objects. How? Well, when using the adapter, although you call `calculateArea()`, it invokes `calculateTriangleArea()`. This approach shows that you can modify the method body as per your needs. For example, in your application, let's assume that each rectangle object has a length of 20 units and a width of 10 units, whereas each triangle object has a base of 20 units and a height of 10 units. So, each rectangle object has an area of $20 \times 10 = 200$ square units and each triangle object has an area of $0.5 \times 20 \times 10 = 100$ square units. Now you can simply multiply each triangle area by 2 to get an equivalent rectangle area and substitute (or use) it where a rectangle area is needed. I believe that this makes sense to you.

Finally, you need to keep in mind that this technique suits best when you deal with objects that are not exactly the same but very similar.

NoteIn the context of the previous point, you should not attempt to convert a circle area to a rectangle area (or do a similar type of conversion), because they are totally different shapes. In this example, I am talking about triangles and rectangles because they have similarities.

Class Diagram

Figure **10-4** shows the class diagram for the important parts of the program. Notice that the Adapter class holds an adaptee (TriInterface) reference.

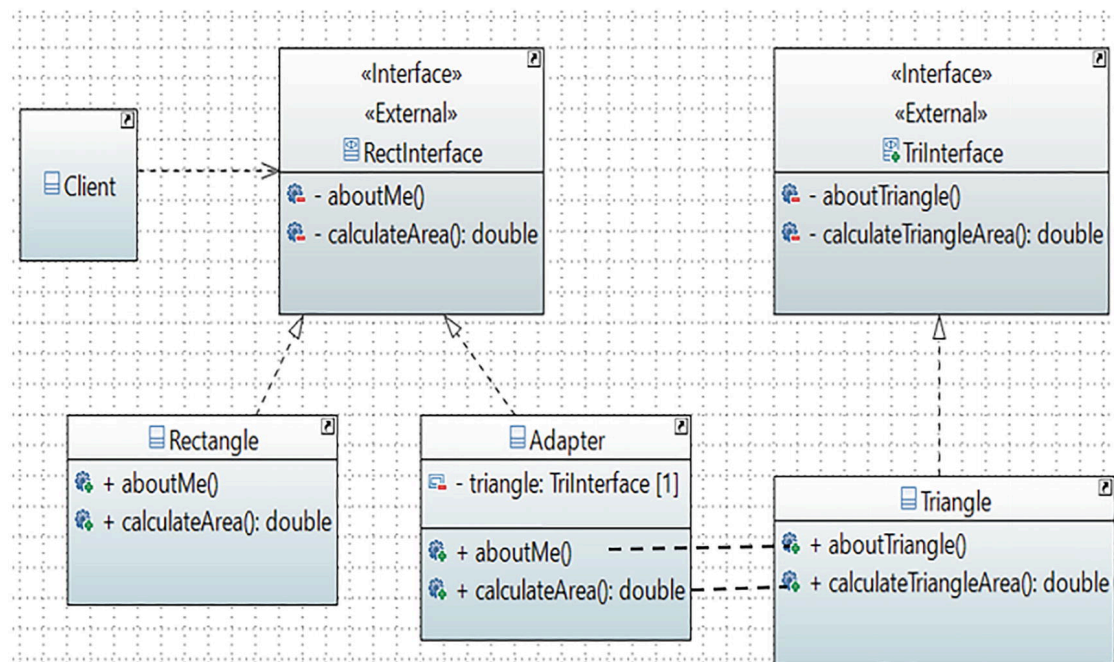


Figure 10-4 Class diagram

Package Explorer View

Figure **10-5** shows the high-level structure of the program.

- ▼  Adapter.java
 - ▼  Adapter
 - ▲ triangle
 - 📄 Adapter(TriInterface)
 - 📄 aboutMe() : void
 - 📄 calculateArea() : double
- >  Client.java
- ▼  Rectangle.java
 - ▼  Rectangle
 - ▲ length
 - width
 - 📄 Rectangle(double, double)
 - 📄 aboutMe() : void
 - 📄 calculateArea() : double
- ▼  RectInterface.java
 - ▼  RectInterface
 - 📄 aboutMe() : void
 - 📄 calculateArea() : double
- ▼  Triangle.java
 - ▼  Triangle
 - ▲ baseLength
 - ▲ height
 - 📄 Triangle(double, double)
 - 📄 aboutTriangle() : void
 - 📄 calculateTriangleArea() : double

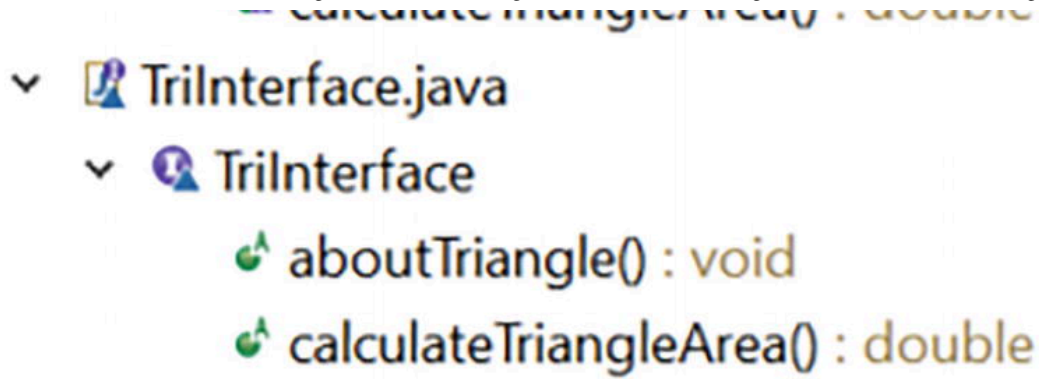


Figure 10-5 Package Explorer view

Demonstration 1

Here's the implementation. All parts of the program are stored in the `jdp3e.adapter` package .

```
// RectInterface.java
interface RectInterface {
    void aboutMe();
    double calculateArea();
}

// Rectangle.java
class Rectangle implements RectInterface {
    double length;
    public double width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    @Override
    public void aboutMe() {
        System.out.println("Shape type: Rectangle.");
    }
    @Override
    public double calculateArea() {
        return length * width;
    }
}

// TriInterface.java
interface TriInterface {
    void aboutTriangle();
    double calculateTriangleArea();
}
```

```
// Triangle.java
class Triangle implements TriInterface {
    double baseLength; // base
    double height; // height
    public Triangle(double length, double height) {
        this.baseLength = length;
        this.height = height;
    }
    @Override
    public double calculateTriangleArea() {
        return 0.5 * baseLength * height;
    }
    @Override
    public void aboutTriangle() {
        System.out.println("Shape type: Triangle.");
    }
}

// Adapter.java
/*
 * Adapter is implementing RectInterface.
 * So, it needs to implement
 * all the methods defined
 * in the target interface.
 */
class Adapter implements RectInterface {
    TriInterface triangle;
    public Adapter(TriInterface triangle) {
        this.triangle = triangle;
    }
    @Override
    public void aboutMe() {
        triangle.aboutTriangle();
    }
    @Override
    public double calculateArea() {
        return triangle.calculateTriangleArea();
    }
}

// Client.java
import java.util.ArrayList;
import java.util.List;
class Client {
    public static void main(String[] args) {
```

```

System.out.println("***Adapter Pattern Demo***\n");
// A rectangle instance
RectInterface rectangle = new Rectangle(20, 10);
// A triangle instance
TriInterface triangle = new Triangle(20, 10);
// Using the adapter for the triangle object
RectInterface adapter = new Adapter(triangle);
// Holding all objects inside a list
// It helps you traverse the list in
// an uniform way.
List<RectInterface> rectangleObjects = new
                                ArrayList<RectInterface>();
rectangleObjects.add(rectangle);
// rectangleObjects.add(triangle); //Error
rectangleObjects.add(adapter); // No error
System.out.println("Processing the following objects:\n");
for (RectInterface rectObject : rectangleObjects) {
    System.out.println("Area: " +
        getDetails(rectObject) + " square units.\n");
}
}
/*
 * The following method does not
 * know whether it gets a rectangle,
 * or a triangle through the adapter.
 */
static double getDetails(RectInterface rectangle) {
    rectangle.aboutMe();
    return rectangle.calculateArea();
}
}

```

Output

Here's the output:

```

***Adapter Pattern Demo***
Processing the following objects:
Shape type: Rectangle.

```

```
Area: 200.0 square units.  
Shape type: Triangle.  
Area: 100.0 square units.
```

Analysis

First, consider the following segment of code:

```
// rectangleObjects.add(triangle); //Error  
rectangleObjects.add(adapter); // No error
```

The commented line says that you cannot directly add the `Triangle` instance into the list of rectangles. But using an adapter, you can do this. This is the real power of using an adapter.

Secondly, note the use of the following code segment with comments inside the client code:

```
/*  
 * The following method does not  
 * know whether it gets a rectangle,  
 * or a triangle through the adapter.  
 */  
static double getDetails(RectInterface rectangle) {  
    rectangle.aboutMe();  
    return rectangle.calculateArea();  
}
```

This segment is optional. I use it to show a usage where you can invoke both methods in one call. Notice that this method was supposed to process the rectangle objects. Now you can see that using an adapter, you can help this method to process a triangle object too.

Types of Adapters

The GoF described two types of adapters: class adapters and object adapters.

Object Adapters

Object adapters adapt through object composition, as shown in Figure 10-6. So, the adapter discussed so far is an example of an object adapter.

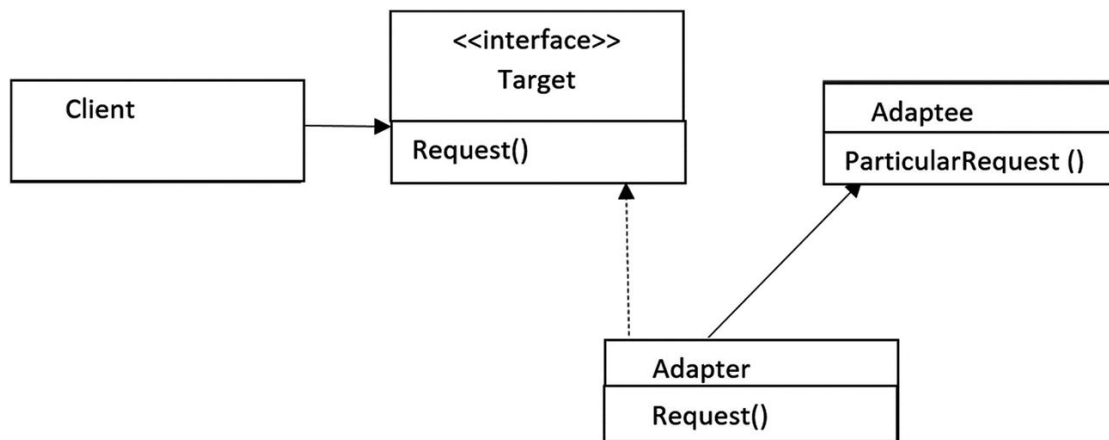


Figure 10-6 Object adapter

In the example, `Adapter` is the adapter that implements `RectInterface` (Target interface). `TriInterface` is the `Adaptee` interface. Notice that the adapter holds the adaptee instance. You can see that it accepts the adaptee as a constructor argument to make use of composition.

Class Adapters

Class adapters adapt through subclassing and support multiple inheritance. But you know that Java does not support multiple inheritance through classes. So, you need interfaces to implement the concept.

Figure 10-7 shows the typical class diagram for class adapters, which support multiple inheritance.

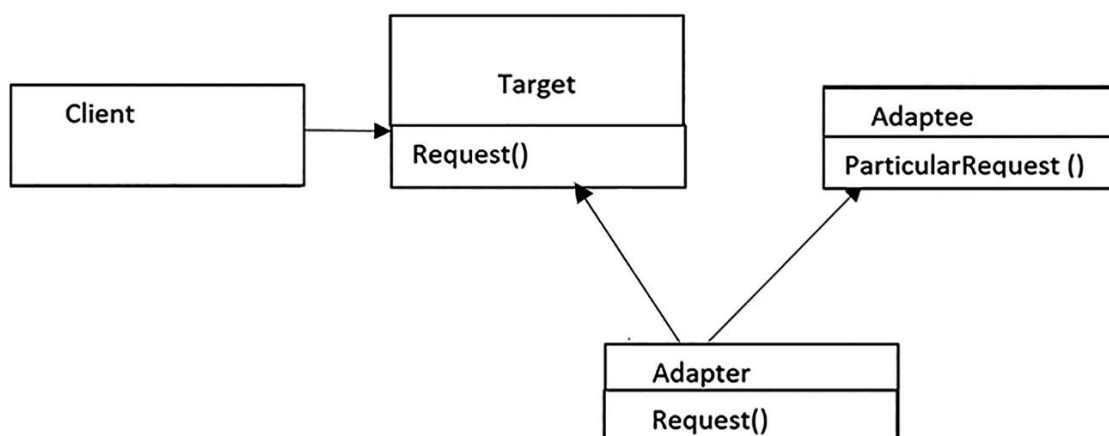


Figure 10-7 Class adapter

Q&A Session

10.1 How can you implement a class adapter design pattern in C#?

You can subclass an existing class and implement the desired interface. Demonstration 2 shows a complete example with output.

Demonstration 2

This demonstration shows the use of a class adapter. To make the example short and simple, I made the `RectInterface` and `TriInterface` interfaces with a single method. `RectInterface` has the method named `aboutMe()`. The `Rectangle` class implements this interface, and thus the following hierarchy is formed:

```
interface RectInterface {  
    void aboutMe();  
}  
class Rectangle implements RectInterface {  
    public void aboutMe() {  
        System.out.println("Shape type: Rectangle.");  
    }  
}
```

`TriInterface` has the method `aboutTriangle()`. The `Triangle` class implements this interface and the following hierarchy is formed:

```
interface TriInterface {  
    void aboutTriangle();  
}  
class Triangle implements TriInterface {  
    public void aboutTriangle(){  
        System.out.println("Shape type: Triangle.");  
    }  
}
```

Now comes the class adapter, which uses the concept of multiple inheritance using a concrete class and an interface. The attached comments will help you understand the code easily.


```
* So, it needs to implement all the methods
* defined in the target interface.
*/
class Adapter extends Triangle implements RectInterface{
    public void aboutMe() {
        // Invoking the adaptee method
        this.aboutTriangle();
    }
}
// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Class Adapter Demo.***\n");
        RectInterface rectangle = new Rectangle();
        System.out.println("Initially, printing the details of
                           both shapes.\n");
        System.out.println("The rectangle.AboutMe() says:");
        rectangle.aboutMe();
        TriInterface triangle = new Triangle();
        System.out.println("The triangle.Triangle() says:");
        triangle.aboutTriangle();
        System.out.println("\nNow using the adapter.");
        RectInterface adapter = new Adapter();
        System.out.println("The adapter.aboutMe() says:");
        adapter.aboutMe();
    }
}
```

Output

Here is the output:

```
***Class Adapter Demo.***
Initially, printing the details of both shapes.
The rectangle.AboutMe() says:
Shape type: Rectangle.
The triangle.Triangle() says:
Shape type: Triangle.
Now using the adapter.
The adapter.aboutMe() says:
Shape type: Triangle.
```

Analysis

This approach may not be suitable in all scenarios. For example, you may need to adapt a method that is not specified in a Java interface. In this case, object adapters are better.

But there is an advantage, too. A class adapter is often called a two-way adapter. Why? Let's add a few lines of code inside the client as follows:

```
class Client {
    public static void main(String[] args) {
        // Previous codes skipped
        // 2-way adapter
        System.out.println("\nTesting a two-way adapter.");
        Adapter adapter2 = new Adapter();
        showAdapter(adapter2);
        showAdaptee(adapter2);
    }
    private static void showAdapter(RectInterface adapter) {
        adapter.aboutMe();
    }
    private static void showAdaptee(TriInterface adaptee) {
        adaptee.aboutTriangle();
    }
}
```

Note the following points:

- You can pass `adapter2` to the method `showAdapter(...)`, which expects a `RectInterface` (Adapter interface) argument from you.
- Again, notice that the `Adapter` class extends from `TriInterface`. It means you can pass `adapter2` to the method `showAdaptee(...)` that expects a `TriInterface` (Adaptee interface) argument.

10.2 Which one do you prefer, class adapters or object adapters?

In most cases, I prefer compositions over inheritances. Object adapters use compositions and are more flexible. Also, in many cases, it is challenging to implement a true class adapter when you need to adapt a specific method from the adaptee interface but there is no close match for it

in the target interface. Apart from this, consider the case when the adaptee class (such as `Triangle` in the example) and the target class are both concrete classes and you cannot inherit from them. In this case, the object adapter is the only solution you have. Lastly, a class adapter can expose some unrelated methods (since you extend from the adaptee and implement the target interface). People can misuse them.

10.3 “...it is challenging to implement a true class adapter when you need to adapt a specific method from an adaptee interface but there is no close match for that in the target interface?” Can you please elaborate?

In my examples, the target interface methods and adaptee interface methods are similar. For example, `RectInterface` has `aboutMe()` and `TriInterface` has `aboutTriangle()`. What are they doing? They are indicating whether it is a rectangle or a triangle. Aren't they? Now suppose there is no such method called `aboutMe()` in `RectInterface` but `aboutTriangle()` still exists in `TriInterface`. In a case like this, if you need to adapt the `aboutTriangle()` method, you need to analyze how to proceed. In this example, `aboutTriangle()` is a simple method but in real-world programming, the method can be much more complex and there can be a dependency associated with that, too. So, when you do not have a corresponding target method, you may find challenges in adapting the method from an adaptee.

10.4 It appears to me that the clients are unaware of the presence of adapters in the code. Is this understanding correct?

Correct. I made this implementation to show you that clients need not know that their requests are translated through an adapter to an adaptee. But if you want to show a message, you could simply add a console message in your adapter in demonstration 2 as follows:

```
class Adapter extends Triangle implements RectInterface {
    public void aboutMe() {
        // Invoking the adaptee method
        // For Q&A
        System.out.println("Using an adapter now.");
    }
}
```

```
        aboutTriangle();  
    }  
}
```

10.5 What happens if the target interface and adaptee interface method signature differ?

It's not a problem at all. If an adapter method has a few parameters, you can invoke the adaptee method with some additional dummy parameters.

In the reverse scenario (if the adapter method has more parameters than the adaptee method), obviously using those additional parameters you can add some functionality before you transfer the call to the adaptee method.

Lastly, when the method parameters are incompatible, you may need to apply the casting mechanism (if possible).

10.6 It appears to me that I can instantiate an adaptee class directly inside the constructor of the object adapter class. Is this correct?

It can work, but it is not a preferred solution. Notice that when you do this, you are promoting tight coupling with the specific adaptee class. But notice this code in demonstration 1:

```
class Adapter implements RectInterface {  
    TriInterface triangle;  
    public Adapter(TriInterface triangle) {  
        this.triangle = triangle;  
    }  
    // The remaining code skipped  
}
```

Using this construct, you can pass any subclass of `TriInterface` to the adapter. This is an efficient solution.

10.7 What are the drawbacks associated with this pattern?

I do not see any big challenges. I believe that an adapter's job is simple and straightforward. You need to write some additional code, but the pay-

off is great, particularly for those legacy systems that cannot be changed but you still want to use them for their stability and simplicity.

I suggest you make adapters as simple as possible. When you use class adapters, you may think that you can override the adaptee behavior. But this practice is discouraged because in this case there is a behavior change between the adapter and the adaptee. It may impose an unnecessary burden when someone needs to debug your code later.

Previous chapter

< [9. Decorator Pattern](#)

Next chapter

[11. Facade Pattern](#) >